AN ADVERSARY-BASED LOWER BOUND FOR SORTING *

Mikhail J. ATALLAH and S. Rao KOSARAJU
Department of Electrical Engineering, The Johns Hopkins University, Baltimore, MD 21218, U.S.A.

Received 27 May 1981; revised version received 24 August 1981

Sorting, adversary arguments, comparison-based

1. Introduction

Several problems exist for which the known time lower bounds are established by nonconstructive means. The sorting problem is a typical example of this class. The existing 'information theoretic' proof [1] of the $\Omega(n \log n)$ lower bound is based on the following argument:

Since the decision tree of any sorting program must have n! leaves corresponding to the n! possible permutations of n distinct numbers, its height must be at least $\log(n!)$, which is $\Omega(n \log n)$.

But suppose we want an actual counter-example to an incorrect sorting program that allegedly 'sorts' in, say, n log log n steps. The information theoretic proof assures us of the existence of an input sequence for which the program produces the wrong output, but it gives us no better way of finding such an input other than trying out all n! possibilities. Here we are interested in 'fast' algorithms, to produce counter examples. Such 'constructive' techniques provide insights into the fine structural complexity of the corresponding problems.

In the following, we will make this notion precise, and then show that the $\Omega(n \log n)$ lower bound for sorting can be established by this technique. Even though adversary arguments are widely used, we were unable to find any source where this notion is made precise. The following definition seems to capture the usual notion of an adversary.

* The research is supported by NSF under grant MCS-79-05163. A preliminary version of this paper was presented at the 1981 CISS, March 1981.

Adversary arguments

Consider any problem X and a class of incorrect programs C, for solving X, such that each program in C has a worst-case running time of O(f(n)) steps on inputs of size n. An *adversary* is a program α, along with a polynomial p, such that α can compute a counter-example to any program P in C within p(f(n)) steps.

Note that the brute force method outlined earlier for producing a counter-example to an n log log n steps sorter is not an adversary since the worst-case speed is (n!)n log log n.

2. An adversary for sorting

In the following we show that an adversary exists for sorting programs that run in O(n log n) steps. We assume that the sorting program gains information about the input numbers by comparisons only. Hence a step of computation is a 'comparison', an 'assignment', a 'print' or a 'halt' operation. The polynomial associated with the adversary turns out to be of degree one. Thus the adversary produces the counter-example in time proportional to the running time of the sorter. The following adversary produces a counter-example for any sorter that takes less than $\frac{1}{4}n \log n$ comparisons. (Throughout, the logarithms are to the base 2).

A correct sorting program must in particular be capable of sorting the n numbers $a_1, a_2, \ldots, a_{n/2}, b_1, b_2, \ldots, b_{n/2}$, where $a_1 < a_2 < \cdots < a_{n/2}$ and $a_i \neq b_j$

0020-0190/81/0000–0000/$02.75 © 1981 North-Holland

for every i and j. We say that b_j *goes into slot* k $(0 \leq k \leq n/2)$ if $a_k < b_j < a_{k+1}$ (assume that $a_0 < b_j < a_{n/2+1}$). Since the a_i's are distinct from each other and from each of the b_j's, any correct sorting program must unambiguously 'determine' into which slot each b_j goes. For each b_j we use a 'range function', R, to keep track of the possible slots into which that b_j can go, and we update this range function after each comparison involving b_j.

More precisely, at any instant in the execution of the program, if $R(j) = (p, q)$ then b_j can go into any one of the slots $p, p+1, \ldots, q$ and still be consistent with the sequence of comparisons and responses undergone by the program up to this stage of execution. Let us denote this as the *consistency property*. The *size* of R(j) is defined to be $q - p + 1$.

When the program terminates, for each j there must be a unique k such that $R(j) = (k, k)$; i.e. the size of each R(j) must equal one.

Before any comparison has been made by the program, there are no constraints on the b_j's. Consequently, initially, the adversary sets $R(j) = (0, n/2)$ for every j. Therefore the initial size of R(j) equals $n/2 + 1$ for every j. We now show that to every comparison performed by the sorting program, the adversary can respond by an outcome and appropriately update the range function so that the sizes of at most two range values are altered, and each of these values gets decreased by a factor of at most 2. Note that a comparison can be of three types: compare (b_{j_1}, b_{j_2}), compare (b_j, a_i), or compare (a_i, a_j).

We first consider comparisons that involve b_{j_1} and b_{j_2}. Assume that before the comparison we had

$$R(j_1) = (\alpha_1, \beta_1) \quad \text{and} \quad R(j_2) = (\alpha_2, \beta_2).$$

In the case analysis that follows the subscripts 1 and 2 are interchangeable.

Case 1. $\beta_1 \leq \alpha_2$. Then the adversary responds by $b_{j_1} \leq b_{j_2}$ and will not adjust any range values.

Case 2. $\alpha_2 \leq \alpha_1 \leq \beta_2 \leq \beta_1$. Then the adversary responds by $b_{j_2} \leq b_{j_1}$, and changes $R(j_1)$ and $R(j_2)$ to

$$R(j_1) = \left(\left\lfloor \frac{\alpha_1 + \beta_2}{2} \right\rfloor, \beta_1\right),$$

and

$$R(j_2) = \left(\alpha_2, \left\lfloor \frac{\alpha_1 + \beta_2}{2} \right\rfloor\right).$$

Case 3.

$$\alpha_1 \leq \alpha_2 \leq \beta_2 \leq \beta_1 \quad \text{and} \quad \left\lfloor \frac{\alpha_1 + \beta_1}{2} \right\rfloor \leq \left\lfloor \frac{\alpha_2 + \beta_2}{2} \right\rfloor.$$

Then the adversary responds by $b_{j_1} \leq b_{j_2}$ and changes $R(j_1)$ and $R(j_2)$ to

$$R(j_1) = \left(\alpha_1, \left\lfloor \frac{\alpha_1 + \beta_1}{2} \right\rfloor\right),$$

and

$$R(j_2) = \left(\left\lfloor \frac{\alpha_1 + \beta_1}{2} \right\rfloor, \beta_2\right).$$

Case 4.

$$\alpha_1 \leq \alpha_2 \leq \beta_2 \leq \beta_1 \quad \text{and} \quad \left\lfloor \frac{\alpha_1 + \beta_1}{2} \right\rfloor > \left\lfloor \frac{\alpha_2 + \beta_2}{2} \right\rfloor.$$

Then the adversary responds by $b_{j_1} > b_{j_2}$ and changes $R(j_1)$ and $R(j_2)$ to

$$R(j_1) = \left(\left\lfloor \frac{\alpha_2 + \beta_2}{2} \right\rfloor + 1, \beta_1\right),$$

and

$$R(j_2) = \left(\alpha_2, \left\lfloor \frac{\alpha_2 + \beta_2}{2} \right\rfloor\right).$$

We next consider comparison that involve some b_j and some a_i. Assume that before the comparison we had $R(j) = (\alpha, \beta)$.

If $i \leq \lfloor (\alpha + \beta)/2 \rfloor$, then the adversary responds by $a_i \leq b_j$, and changes R(j) to: $R(j) = (\lfloor (\alpha + \beta)/2 \rfloor, \beta)$. Otherwise, the adversary responds by $a_i > b_j$, and changes R(j) to: $R(j) = (\alpha, \lfloor (\alpha + \beta)/2 \rfloor)$.

For any comparison of the form compare (a_i, a_j), the adversary responds by $a_i \leq a_j$ if $i \leq j$, and by $a_i > a_j$ if $i > j$.

It is easy to see that the consistency property of the range values is preserved after every comparison. Moreover in the worst case (for the adversary) the size of two range values are divided by two. Since initially all n/2 range values have size $n/2 + 1$, and since their sizes have to equal one when the correct sorting program produces an answer, it follows that $(n/4)\log n$ is a lower bound on the number of comparisons needed to achieve this.

If a 'sorter' takes less than $(n/4)\log n$ comparisons, then just before the sorter is about to output the numbers, at least one range value has size greater than

1. Let R(k) be (p, q), where $p \neq q$. At this stage, the adversary picks two input sequences both 'satisfying' the consistency constraints for b_i's, the only difference between the sequences being in the values of b_k. In the first sequence $a_p < b_k < a_{p+1}$, and in the second sequence $a_{p+1} < b_k < a_{p+2}$. The sorter will incorrectly order a_{p+1} and b_k for at least one of the sequences. Based on the sorter's output, the adversary can pick one of the two sequences as the desired counter-example. It can be easily verified that the adversary runs in O(f(n)) steps, where f(n) is the speed of the sorter, if f(n) is $\Omega(n)$.

The constant factor multiplying n log n can be improved from 1/4 to 1/2, by considering the input sequence $a_1 a_2 \cdots a_{n/\log n} b_1 b_2 \cdots b_{n-n/\log n}$. The details are trivial, and will be skipped.

3. Open problem

Another problem that may be amenable to an adversary argument is the problem of finding the k-th smallest element from a large heap [3].

Given a heap of depth greater than k, with the smallest element at the root, we are interested in finding the k-th smallest element in the heap. An algorithm exists [3] for solving this problem in O(k log k) steps. Can a lower bound of $\Omega(k \log k)$ be established for this problem ?

Acknowledgements

Professor Janos Simon [4] brought to our attention the open problem of finding an adversary argument for sorting.

References

[1] A.V. Aho, J.E. Hopcroft and J.D. Ullman, The Design and Analysis of Computer Algorithms (Addison-Wesley, Reading, MA, 1974).
[2] M.J. Atallah and S.R. Kosaraju, An adversary-based lower bound for sorting, Proc. 15th Annual Conf. on Information Sciences and Systems (1981) pp. 7–8.
[3] D.B. Johnson, personal communication.
[4] Janos Simon, personal communication.